Docker Notes for Linux Setup (Using Containers Instead of VMs)

Docker allows you to run lightweight **containers** instead of full VMs. Below are the most useful concepts and commands.

---

1. `docker run` – Start a New Container

```
docker run [OPTIONS] IMAGE [COMMAND]
```

Common Options:

- `-d` → Detached mode (runs in background)
- `-i` → Interactive (keeps STDIN open)
- `-t` → Allocate a pseudo-TTY (gives you a terminal)
- `-p HOST:CONTAINER` → Map host port → container port
- `--name` → Assign a name to the container
- `--hostname` → Set container hostname
- `--restart unless-stopped` → Auto-restart policy
- `--cpus` → Limit CPU usage
- `--memory` → Limit memory usage
- `--env` → Set environment variables

Example (Ubuntu container):

```
docker run -dit -p 8080:80 ubuntu:latest /bin/bash
```

Explanation: - Starts a new container from `ubuntu:latest` - Runs in background (`-d`) with interactive TTY (`-it`) - Maps **host port 8080 → container port 80** - Runs `/bin/bash` as the main process (keeps container alive)

👉**Result:** A new container is created and runs in background. You can later `exec` into it.

---

2. Difference Between `run` and `exec`

- `docker run` → Always creates a **new container** from an image.
- `docker exec` → Runs a command inside an **already running container**.

Example:

```
# Start new container from image
docker run -it ubuntu:latest /bin/bash

# Open a new shell inside an existing container
docker exec -it ubuntu-container /bin/bash
```

👉`run` = build a new house. `exec` = enter an existing house.

---

3. Container Management Commands

```
docker ps            # List running containers
docker ps -a         # List all containers (running + stopped)
docker images        # List downloaded images

docker stop <id>     # Stop a container
docker start <id>    # Start a stopped container
docker rm <id>       # Remove a stopped container
docker rmi <image>   # Remove an image
```

---

4. Access a Container Shell

```
docker exec -it <container_id_or_name> /bin/bash
```

Examples:

```
docker exec -it ubuntu-container /bin/bash
docker exec -it ubuntu-container date
docker exec -it ubuntu-container ls -l /
docker exec -it ubuntu-container echo "Hello from inside"
```

👉Runs commands inside the container **without creating a new one**.

---

5. Why `/bin/bash` at the End?

- `/bin/bash` is the **main process (PID 1)** inside the container.
- If not specified, the container may exit immediately (no process running).
- Exiting from `/bin/bash` = container stops.

---

6. Advanced Examples

Ubuntu container with limits & env vars:

```
docker run -dit
  --name ubuntu-container
  --hostname ubuntu-dev
  --restart unless-stopped
  --cpus="2"
  --memory="4g"
  -p 8080:80
  --env TZ=Asia/Kolkata
  --env LANG=en_US.UTF-8
  ubuntu:latest /bin/bash

# Access it
docker exec -it ubuntu-container /bin/bash
```

Red Hat UBI (Universal Base Image):

```
docker run -dit
  --name myredhat
  -p 8080:80
  --memory="512m"
  --cpus="1.5"
  registry.access.redhat.com/ubi8/ubi:latest
  /bin/bash

# Access it
docker exec -it myredhat /bin/bash
```

Or in two steps:

```
docker pull registry.access.redhat.com/ubi8/ubi
docker run -it registry.access.redhat.com/ubi8/ubi bash
```

👉Drops you directly into the container shell.

---

7. When to Use `-it` vs `-d`

- `-it` → Interactive terminal session (e.g., bash, mysql client)
- `-d` → Run in background (e.g., web server, database)

If you run `-d` with bash, the container runs silently in background but you can't interact with it.

Quick Reference

Command Example	Purpose
`docker run -it ubuntu bash`	Start new container + interactive shell
`docker exec -it mycontainer bash`	Enter shell of running container
`docker ps`	List running containers
`docker stop <id>`	Stop container
`docker start <id>`	Restart stopped container
`docker rm <id>`	Remove container
`docker rmi <image>`	Remove image

👈Use **Docker containers** as lightweight Linux environments instead of VMs.